Memory System

Table of Contents

- 1. Overview and Design Philosophy
  - 1.1 Core Design Principles
  - 1.2 Principle 2: Four-Point Timestamp Anchoring
  - 1.3 Principle 3: Side Query
  - 1.4 Principle 4: TRUSTING_RECALL_SECTION
  - 1.5 Principle 5: Strict Scope Isolation
  - 1.6 extractMemories Mutual Exclusion Design
  - 1.7 Why Plain Text Instead of a Database (Speculation)
- 2. File Structure Overview
  - 2.1 Module Directory Structure
- 3. Memory Type System in Detail
  - 3.1 Four Type Definitions
  - 3.2 Scope and Save Specifications per Type
  - 3.3 Frontmatter Format Specification
  - 3.4 Memory Drift Warning
  - 3.5 Parsing Functions
- 4. Storage Mechanism: File System Architecture
  - 4.1 Directory Structure (Three-Layer Scope)
  - 4.2 MEMORY.md Index File (Entrypoint)
  - 4.3 Auto Memory Directory Path Resolution
  - 4.4 Team Memory Path
- 5. Core Data Structures
  - 5.1 MemoryHeader (Scan Result)
  - 5.2 RelevantMemory (Retrieval Result)
  - 5.3 MemoryFileInfo (Complete Memory File)
  - 5.4 SyncState (Team Memory Sync State)
  - 5.5 SyncResult Various Response Types
  - 5.6 SessionMemoryConfig (Session Memory Configuration)
- 6. Retrieval and Search Algorithm
  - 6.1 Memory File Scanning (scanMemoryFiles)
  - 6.2 AI Relevance Retrieval (findRelevantMemories)
  - 6.3 Conditional Rules
- 7. Memory Staleness Mechanism
  - 7.1 Algorithm Implementation
  - 7.2 Trigger Conditions
  - 7.3 Design Intent
- 8. Session Memory System
  - 8.1 System Positioning

    - 8.2 Extraction Trigger Conditions
    - 8.3 Session Memory Template Structure
    - 8.4 Extraction Execution Flow
    - 8.5 Session Memory Compaction
- 9. Team Memory Cloud Sync System
    - 9.1 API Endpoints
    - 9.2 Sync State Machine
    - 9.3 Full Sync Algorithm
    - 9.4 Hash Algorithm
    - 9.5 File System Watcher
    - 9.6 Secret Scanner
- 10. Agent Memory System
    - 10.1 Three Scopes
    - 10.2 Snapshot System
- 11. @include Directive in Detail
    - 11.1 Syntax
    - 11.2 Parsing Flow (claudemd.ts)
    - 11.3 Supported File Types
    - 11.4 Conditional @include (paths field)
- 12. Memory Loading Order and Priority
    - 12.1 Four-Level Loading Order
    - 12.2 Content Merge Strategy
- 13. Security Mechanisms
    - 13.1 Path Traversal Protection
    - 13.2 Configuration Injection Protection
    - 13.3 Sensitive Information Protection
    - 13.4 File Permissions
- 14. Complete Index of Key Functions
    - 14.1 Auto Memory Module (memdir/)
    - 14.2 Session Memory Module (services/SessionMemory/)
    - 14.3 Team Memory Sync Module (services/teamMemorySync/)
    - 14.4 Agent Memory Module (tools/AgentTool/)
    - 14.5 Core Utilities Module (utils/)
    - 14.6 Auto-Dream Module (services/autoDream/)
    - 14.7 Memory Extraction Module (services/extractMemories/)
- 15. System Architecture Overview
    - 15.1 Architecture Main Diagram
    - 15.2 Background Process Diagram
- 16. Algorithm Complexity Analysis
    - 16.1 Operation Complexity Table
- 17. Auto-Dream System (Automatic Memory Consolidation)
    - 17.1 System Positioning

- 17.2 Lock File Mechanism (consolidationLock.ts)
- 17.3 Four-Phase Consolidation Prompt (consolidationPrompt.ts)
- 17.4 Tool Constraints
- 17.5 DreamTask UI Integration
- 17.6 GrowthBook Configuration and Feature Flag
- 17.7 Relationship with /dream Skill
- 18. ExtractMemories System (End-of-Session Memory Extraction)
  - 18.1 System Positioning
  - 18.2 Forked Agent Mode
  - 18.3 Closure State and Trailing Run
  - 18.4 Submission Pipeline
  - 18.5 Extraction Prompt Template
- 19. DreamTask (Dream Task UI Integration)
  - 19.1 Design Background
  - 19.2 Task State Structure
  - 19.3 Message Listener (makeDreamProgressWatcher)
  - 19.4 Kill Task
- 20. Daily Log Mode (Assistant Mode / KAIROS)
  - 20.1 Background
  - 20.2 Differences from Standard Mode
- 21. Startup Chain and Module Initialization
  - 21.1 backgroundHousekeeping.ts
  - 21.2 stopHooks.ts Trigger Chain
  - 21.3 Drain on Shutdown
- Appendix A: Zod Schema Definitions
  - A.1 Schema Definitions
- Appendix B: Global State Overview
  - B.1 Global State per Module

---

1. Overview and Design Philosophy

Claude Code's memory system is a **layered, file-based, long-term persistent** architecture, using plain text (Markdown + YAML frontmatter) as the storage medium, not a database.

> **Note**
>
> YAML frontmatter = the "index card" of a memory
>
> - name, description, type — used to find this memory and determine relevance
> Markdown body = the "body content" of a memory
> - the actual information to remember
>
> Relationship: frontmatter is metadata attached to the top of a Markdown file; both are written in the same .md file

	YAML frontmatter	Markdown body
目的	检索、分类、相关性判断	存储实际内容
格式	结构化键值对	自然语言文本
使用时机	加载时快速扫描、 relevance 过滤	命中介入时才作为上下文读入

1.1 Core Design Principles

Principle 1: Do not save derivable information -> i.e., information that can be directly recovered

The following are explicitly defined as "should not be stored in memory":

- Code patterns, conventions, architecture (derivable from the project's current state)
- Git history (use `git log` / `git blame`)
- Debugging solutions (the fix code itself is authoritative)
- Content already in CLAUDE.md
- Ephemeral task details

> **Eval-validated supplement in source code**: In `memoryTypes.ts`, the last paragraph of `WHAT_NOT_TO_SAVE_SECTION`:
>
> ```
> // H2: explicit-save gate. Eval-validated (memory-prompt-iteration case 3,
> // 0/2 → 3/3): prevents "save this week's PR list" → activity-log noise.
> 'These exclusions apply even when the user explicitly asks you to save.
>  If they ask you to save a PR list or activity summary,
>  ask what was *surprising* or *non-obvious* about it — that is the part worth keeping.'
> ```
>
> This is an **eval-driven design decision**: without this rule, the model would directly execute "save what the user asked to save," resulting in activity-log noise. Core insight: **User asking to save ≠ should save** — only the insight *behind* what to save is worth keeping.

1.2 Principle 2: Four-Point Timestamp Anchoring

Each memory carries an `mtimeMs` (modification time) in the file, and the system calculates freshness from it. Memories older than 1 day trigger a "staleness warning," requiring the user to verify the current code state.

```
// memoryAge.ts
export function memoryAgeDays(mtimeMs: number): number {
  return Math.max(0, Math.floor((Date.now() - mtimeMs) / 86_400_000))
}

export function memoryFreshnessText(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d <= 1) return ''                          // today/yesterday → no notice
  if (d <= 7) return `This memory is ${d} days old.`
  if (d <= 30) return `This memory is ${d} days old. Verify before citing.`
  // 30+ days → stale reasoning text
}
```

> **Design rationale** from `memoryAge.ts` comments:
> *"Models are poor at date arithmetic — a raw ISO timestamp doesn't trigger*

> *staleness reasoning the way '47 days ago' does."*
>
> Language models have poor arithmetic reasoning on ISO timestamps. Giving "47 days ago" directly is more effective at triggering the model's awareness of stale memory than giving "2026-02-12T...". Additionally, the warning copy deliberately emphasizes "point-in-time observation" — **a memory is a snapshot of state at a point in time, not current state** — which is especially important for memories containing file:line references (those citations may have long since become invalid).

1.3 Principle 3: Side Query

The relevance retrieval uses the **Sonnet model for an independent query**, consuming no main conversation context tokens and not affecting the main conversation's reasoning flow.

```
// SELECT_MEMORIES_SYSTEM_PROMPT key design excerpt
const SELECT_MEMORIES_SYSTEM_PROMPT = `...
- If a list of recently-used tools is provided, do not select memories that
  are usage reference or API documentation for those tools
  (Claude Code is already exercising them).
  DO still select memories containing warnings, gotchas, or known issues
  about those tools — active use is exactly when those matter.`
```

There is a precise semantic distinction here: **tool usage documentation is NOT selected** (it's already in the conversation), but **tool "warnings/gotchas/known issues" ARE selected** — during active use, warning information is most useful right now. Through the `recentTools` parameter, the selector excludes those tools' API reference docs while retaining warning-type memories in active usage scenarios.

> **Telemetry hook** (`findRelevantMemories.ts` , controlled by `feature('MEMORY_SHAPE_TELEMETRY')`): The model's selection results and scan results are sent to `logMemoryRecallShape` , recording metrics like `selection_rate` . **Data is sent even when the selection set is empty** — the denominator (`memories.length`) always exists, making it possible to distinguish "retrieval ran but nothing was selected" from "retrieval didn't run at all" — two completely different product signals.

Note

Is memory retrieval async? Could there be timing issues when loading into context?

> Answer: This prefetch mode makes a pragmatic trade-off between "non-blocking" and "trying to complete within the current round" — using polling instead of blocking wait, allowing the prefetch to start later than the API call, but doing its best to complete before the main LLM's streaming response in this round (**It's very likely it won't make it before the API request is sent, but it tries to catch up before the main LLM's streaming returns in this round**). To strictly guarantee "retrieval must be synchronous," the prefetch would need to be changed to await before the for await, but this introduces an extra RTT delay.

1.4 Principle 4: TRUSTING_RECALL_SECTION — Trust Levels After Memory Recall

This is an **eval-validated design** in `memoryTypes.ts` . The core idea: **finding a memory ≠ being able to trust its content**.

```
// memoryTypes.ts
export const TRUSTING_RECALL_SECTION: readonly string[] = [
  '## Before recommending from memory',
  'A memory that names a specific function, file, or flag is a claim that ' +
    'it existed *when the memory was written*. It may have been renamed, ' +
    'removed, or never merged. Before recommending it:',
  '- If the memory names a file path: check the file exists.',
  '- If the memory names a function or flag: grep for it.',
  '- If the user is about to act on your recommendation, verify first.',
  '"The memory says X exists" is not the same as "X exists now."',
]
```

1.5 Principle 5: Strict Scope Isolation

Each of the four memory types has its implied default scope (private or team), and the configuration injection path has security restrictions to prevent malicious project configurations from writing memories to sensitive directories.

1.6 extractMemories Mutual Exclusion Design — Main Agent and Forked Agent Division

The core design philosophy of `extractMemories.ts`: **not "two systems competing," but "main agent writes first, forked agent fills in the gaps."**

```
// extractMemories.ts
/**
 * Returns true if any assistant message after the cursor UUID contains a
 * Write/Edit tool_use block targeting an auto-memory path.
 *
 * The main agent's prompt has full save instructions — when it writes
 * memories, the forked extraction is redundant. runExtraction skips the
 * agent and advances the cursor past this range, making the main agent
 * and the background agent mutually exclusive per turn.
 */
function hasMemoryWritesSince(
  messages: Message[],
  sinceUuid: string | undefined,
): boolean {
  // ...checks if the main agent has written to auto-memory paths
}
```

Collaborative flow:

1. The main agent **actively wrote memories** in the conversation (guided by full system prompt instructions) → the forked agent detects this, **skips extraction but advances the cursor** → the main agent wins.
2. The main agent **did not write memories** in the conversation → the forked agent kicks in, uses an independent extraction prompt to analyze recent conversation and add memories.

> **Why this design?** The main agent's system prompt has complete memory-saving instructions; what it writes is "intentional" (based on current conversation semantics). The forked agent is "indiscriminate scanning"

> (structured extraction from the last N messages). Each has advantages: **the main agent writes insights** (why this was done), **the forked agent fills in facts** (what happened). Only together do they form a complete memory.

Throttle + Trailing Run double protection:

```
// extractMemories.ts
// Only run extraction once every N eligible turns (tengu_bramble_lintel, default 1)
turnsSinceLastExtraction++
if (turnsSinceLastExtraction < getFeatureValue_CACHED('tengu_bramble_lintel', 1)) {
  return  // skip this run
}

// If a call arrives while one is already running, stash the context for a trailing run
if (inProgress) {
  pendingContext = { context, appendSystemMessage }  // trailing run
  return
}

// trailing run: process messages accumulated between two calls
const trailing = pendingContext
if (trailing) {
  await runExtraction({ context: trailing.context, isTrailingRun: true })
}
```

- **Trailing Run design**: During the main extraction run, the user may have sent several more messages. Stash the latest context, and after the current extraction completes, immediately run a trailing with the new messages, ensuring nothing is missed.
- **Throttle design** (`tengu_bramble_lintel`): Avoid triggering extraction every turn; default value is 1 (may trigger every turn, but constrained by other conditions).

5-turn hard cap: The extraction agent is force-terminated after running at most 5 turns.

> Source code comment: `"Well-behaved extractions complete in 2-4 turns (read → write). A hard cap prevents verification rabbit-holes from burning turns."`
>
> Prevents the agent from falling into a "read → verify → re-read → re-verify" rabbit hole, consuming excessive tokens. In practice, 2-4 turns are sufficient for a valid extraction.

"Single-pass read-then-sort" optimization for memory scanning (`scanMemoryFiles.ts`):

```
// memoryScan.ts
/**
 * Single-pass: readFileInRange stats internally and returns mtimeMs, so we
 * read-then-sort rather than stat-sort-read. For the common case (N ≤ 200)
 * this halves syscalls vs a separate stat round; for large N we read a few
 * extra small files but still avoid the double-stat on the surviving 200.
 */
```

- Common case (N ≤ 200): read only the first 30 lines of frontmatter, one traversal
- Large N case: read at most 200 file frontmatters, still only 200 reads (not stat all files then select 200).
- `readFileInRange` internally returns both content and mtime, so stat is integrated into the read path.

"Pre-commitment" design for ensureMemoryDirExists (`memdir.ts`):

```
// memdir.ts
/**
 * Shared guidance text appended to each memory directory prompt line.
 * Shipped because Claude was burning turns on `ls`/`mkdir -p` before writing.
 * Harness guarantees the directory exists via ensureMemoryDirExists().
 */
export const DIR_EXISTS_GUIDANCE =
  'This directory already exists — write to it directly with the Write tool ' +
  '(do not run mkdir or check for its existence).'
```

Design intent: Without pre-creating the directory, the model would spend 1-2 turns checking/creating the directory before writing memories (burning turns). By calling `ensureMemoryDirExists` (guaranteed in the `loadMemoryPrompt` entry point) to create the directory in advance, and then directly telling the model in the prompt that "the directory already exists, no need for mkdir," the saved turns can be used for actual work.

1.7 Why Plain Text Instead of a Database (Speculation)

1. Naturally compatible with Git workflows, achieving "code and cognition" version alignment
Plain text files naturally support Git diff comparison, version history tracking, and PR review. If using a database, Git can only store binary diffs, making meaningful code review and change comparison impossible

2. Human readability and intervenability
Using plain text format, developers can open files directly in an editor, read, correct errors, or supplement context just like modifying regular code

3. Fully aligned with the Agent's cognitive model and native capabilities
Plain text (such as Markdown) is the format large language models are best at processing. Give it a path, and it can directly read, write, and edit. If a database is introduced, the Agent also needs to master additional capabilities like database connections, table creation, schema migration, SQL queries, etc., increasing unnecessary complexity

4. No additional infrastructure or deployment costs, lowering cross-environment sync barriers
Plain text files depend on no external services, making them the simplest and lightest persistence solution. In team collaboration, **plain text memory files can be directly pulled to new members' local environments with the code repository, achieving true "out-of-the-box" functionality.**

5. Thanks to the popularization of underlying models with ultra-long context windows

2. File Structure Overview

2.1 Module Directory Structure

```
src/
├── memdir/                                # ★ Auto Memory Core Module
│   ├── memoryTypes.ts                     # Memory type enum, frontmatter parsing
│   ├── memoryScan.ts                      # Memory file scanning, manifest formatting
│   ├── memoryAge.ts                       # Memory freshness calculation
│   ├── memdir.ts                          # Memory prompt construction, MEMORY.md
truncation
│   ├── paths.ts                           # Memory path resolution, auto memory toggle
```

```
│   ├── teamMemPaths.ts                     # Team memory path security validation
│   ├── teamMemPrompts.ts                   # Team memory prompt composition
│   └── findRelevantMemories.ts             # AI relevance retrieval
│
├── services/
│   ├── SessionMemory/                      # ★ Session Memory Module
│   │   ├── sessionMemory.ts               # Extraction trigger logic, REPL hook
│   │   ├── sessionMemoryUtils.ts           # Global state, config, wait functions
│   │   └── prompts.ts                     # Session memory template, update prompt
│   ├── teamMemorySync/                     # ★ Team Memory Cloud Sync Module
│   │   ├── index.ts                       # pull/push/sync main logic
│   │   ├── watcher.ts                     # fs.watch debounced push
│   │   ├── secretScanner.ts               # Pre-upload sensitive info scan
│   │   └── types.ts                       # Zod schema, API response types
│   ├── compact/
│   │   └── sessionMemoryCompact.ts        # Context compression based on session memory
│   ├── autoDream/                          # ★ Auto-Dream Background Consolidation Module
│   │   ├── autoDream.ts                   # Three-gate trigger + forked agent execution
│   │   ├── config.ts                      # isAutoDreamEnabled() toggle
│   │   ├── consolidationLock.ts           # Lock file (mtime = lastConsolidatedAt)
│   │   └── consolidationPrompt.ts         # Four-phase consolidation prompt
│   └── extractMemories/                    # ★ End-of-Session Memory Extraction Module
│       ├── extractMemories.ts              # Forked agent extraction main logic
│       └── prompts.ts                      # Extraction prompt template
│
├── tasks/DreamTask/
│   └── DreamTask.ts                        # DreamTask UI integration (footer pill)
│
├── tools/AgentTool/
│   ├── agentMemory.ts                      # Agent-level persistent memory
│   └── agentMemorySnapshot.ts              # Snapshot initialization and sync
│
├── components/memory/
│   ├── MemoryFileSelector.tsx              # Memory file selection UI component
│   └── MemoryUpdateNotification.tsx       # Memory update notification component
│
├── commands/memory/
│   └── memory.tsx                          # /memory CLI command implementation
│
└── utils/
    ├── claudemd.ts                         # CLAUDE.md loading main logic, @include
parsing
    └── memoryFileDetection.ts              # Memory file type auto-detection
```

---

3. Memory Type System in Detail

3.1 Four Type Definitions

Defined in `src/memdir/memoryTypes.ts`, using **const assert closed-set enum**:

```
export const MEMORY_TYPES = [
  'user',       // User preferences, role, expertise
```

```
  'feedback',   // Work guidance (behaviors to avoid/repeat)
  'project',    // Ongoing work, goals, initiatives
  'reference'   // References/pointers to external systems
] as const

export type MemoryType = (typeof MEMORY_TYPES)[number]
// Derived result: 'user' | 'feedback' | 'project' | 'reference'
```

3.2 Scope and Save Specifications per Type

Type	Default Scope	When to Escalate to Team	Example Save Content
`user`	Private (Always private)	Never	User is a senior Go engineer, first time touching React
`feedback`	Private (default)	Only when team consensus needed	Integration tests must connect to real database (mock caused a production incident)
`project`	Team (strongly biased)	Usually team	Linear project "INGEST" tracks all pipeline bugs
`reference`	Team (usually)	Usually team	Grafana dashboard grafana.internal/d/api-latency is the oncall monitoring board

> **Note**
>
> Private memories (user / feedback / project / reference) only exist on local disk; only **Team Memory** (team-shared portion) syncs to the cloud for sharing among organization members of the same repo

3.3 Frontmatter Format Specification

Each memory is a Markdown file and must include YAML frontmatter:

> **Note**
>
> Uses standard YAML frontmatter (--- delimiters), parsed directly with regex

```
---
name: {{unique memory name — use snake_case or kebab-case}}
description: {{one-line description — used by AI to judge relevance, the more specific the
better}}
type: {{user | feedback | project | reference}}
---

Memory body content.

For feedback/project types, recommended format:
Rule/fact itself
**Why:** Reason explanation — usually a past event or strong preference
**How to apply:** When and where to apply this rule
```

> **YAML special character auto-quoting design** (`frontmatterParser.ts`):
>
> Source code comments explain why this design is needed:

```
const YAML_SPECIAL_CHARS = /[{}[]*&#!|>%@`]|: /
// - { } are flow mapping indicators
// - * is anchor/alias indicator
// - [ ] are flow sequence indicators
// - ': ' (colon followed by space) is key indicator
//   causes 'Nested mappings are not allowed in compact mappings'
// - # is comment indicator
```

> Typical problem scenario: `paths: "**\\/*.{ts,tsx}"` contains `*`, `{`, `}`, `/` — all YAML special characters. If not handled, writing directly into frontmatter will cause YAML parser errors.
>
> `quoteProblematicValues()` automatically retries on parse failure, wrapping all values with special characters in double quotes. Values are also pre-quoted on write, ensuring round-trip consistency.

3.4 Memory Drift Warning

When memory content conflicts with current code state, the system requires: (Current situation has higher priority than memory per recency)

> Trust what you observe now, not what's recorded in memory — update or delete stale memories, do not make assertions based on outdated information.

3.5 Parsing Functions

```
// memoryTypes.ts
export function parseMemoryType(raw: unknown): MemoryType | undefined {
  // Invalid or missing values return undefined, no exception thrown
  if (typeof raw !== 'string') return undefined
  // Second line: exact match in ['user', 'feedback', 'project', 'reference']
  return MEMORY_TYPES.find(t => t === raw)
}
```

---

4. Storage Mechanism: File System Architecture

4.1 Directory Structure (Three-Layer Scope)

```
~/.claude/                                  # User-level (global private)
├── memory/
│   ├── MEMORY.md                           # Index manifest (entry point)
│   ├── user_role.md                        # Memory files *.md
│   ├── feedback_testing.md
│   ├── project_ingest.md
│   ├── reference_linear.md
│   └── team/                               # Team-shared memory
│       ├── MEMORY.md
│       ├── *.md
│
├── agent-memory/                           # Agent persistent memory (user scope)
│   └── <agentType>/
│       └── MEMORY.md
│
└── agent-memory-local/                     # Agent persistent memory (local scope)
```

```
    └── <agentType>/
        └── MEMORY.md

<git-root>/.claude/                          # Project-level (shared within project)
├── memory/
│   ├── MEMORY.md
│   ├── *.md
│   └── team/
│       ├── MEMORY.md
│       └── *.md
│
└── agent-memory/                             # Agent persistent memory (project scope)
    └── <agentType>/
        └── MEMORY.md
```

Distinction:

```
~/.claude/projects/<project>/memory/ — 用户 Auto Memory

- 作用域: 用户 × 项目 (跨会话)
- 路径: ~/.claude/projects/<sanitized-git-root>/memory/
- 性质: 用户个人的记忆, 存在于用户配置目录下
- 可见性: 只有该用户自己能看到
- 团队共享: 不共享 (不在项目目录内)
- 记忆范围: user / feedback / project / reference 四类, 这是 prefetch 和 findRelevantMemories 操作的主要目录

~/.claude/agent-memory/<agentType>/ — Agent Memory (user scope)

- 作用域: agent × 用户全局 (跨所有项目)
- 路径: ~/.claude/agent-memory/<agentType>/
- 性质: agent 的持久化记忆, 跟 agent 类型走, 不跟项目走
- 可见性: 只有该 agent 自己能看到
- 团队共享: 不共享

<cwd>/.claude/agent-memory/<agentType>/ — Agent Memory (project scope)

- 作用域: agent × 当前项目 (跨会话)
- 路径: 在项目根目录的 .claude/agent-memory/<agentType>/
- 性质: agent 记忆, 跟项目走, 会被提交到 git
- 可见性: 团队所有成员共享同一 agent 实例的上下文时可见

<cwd>/.claude/agent-memory-local/<agentType>/ — Agent Memory (local scope)

- 作用域: agent × 当前机器 × 当前项目
- 路径: 在项目目录, gitignored (.claude 通常在 gitignore 里)
- 性质: 本地专用, 不同步到 VCS
- 可见性: 只有当前机器的当前项目能访问
```

4.2 MEMORY.md Index File (Entrypoint)

Nature: **MEMORY.md is an index manifest, not a memory container.** This differs from other systems (like Obsidian) — content should not be written directly into MEMORY.md.

Format specification:

```
- [Title](filename.md) — one-line description (optional)
- [user] user_role.md (2026-03-30T10:00:00.000Z): User is a data scientist
- [feedback] feedback_testing.md (2026-03-29T08:00:00.000Z)
```

Hard limits (`memdir.ts`):

- Maximum 200 lines
- Maximum 25,000 bytes

When limits are exceeded, `truncateEntrypointContent()` applies both line count and byte count truncation simultaneously, recording `wasLineTruncated` and `wasByteTruncated` flags.

After truncation, a warning is appended:

```
> WARNING: MEMORY.md is 347 lines and 94KB (limit: 200 lines and 25KB).
    Only part of it was loaded. Keep index entries to one line under ~200 chars;
    move detail into topic files.
```

> **Note**
>
> Information is not completely lost — only the index in MEMORY.md is lost. So even if an entry is truncated, if the user asks about related content, the Sonnet relevance selector can still read that topic file's `frontmatter` from disk and retrieve it.

4.3 Auto Memory Directory Path Resolution

Full path resolution (`getAutoMemPath()` , priority high to low):

```
  1. CLAUDE_COWORK_MEMORY_PATH_OVERRIDE (env var) → returns full path directly, no joining
  2. autoMemoryDirectory config item (settings.json, priority order: policySettings >
flagSettings > localSettings > userSettings)
    - ⚠  Project-level .claude/settings.json is prohibited from setting this field (security
protection: prevents malicious repos from redirecting memory path to
  ~/.ssh or other sensitive directories)
  3. Default path:
  <memoryBase>/projects/<sanitized-git-root>/memory/
    - sanitizePath(): converts git root path to safe directory name (non-alphanumeric chars
replaced with -, long names truncated with hash suffix)
    - getAutoMemBase(): uses git root instead of CWD, ensuring all worktrees of the same
repo share the same memory directory

  Collaboration sharing:
  - All worktrees under the same git root share the same auto memory
  - Team memory additionally adds <autoMemPath>/team/ directory on top (via TEAMMEM feature
branch)
```

4.4 Team Memory Path

`src/memdir/teamMemPaths.ts` — `getTeamMemPath()` :

```
<memoryBase>/projects/<sanitized-project-root>/memory/team/
```

> **Note**
>
> Team memory refers to server-side state information and similar data, meaning public information accessible to authorized people

```
Team Memory Sharing Mechanism Summary

1. Core Definition and Granularity
Team Memory aims to share memories among all collaborators of the same Git repository.
Identification granularity: git remote URL (owner/repo) as the unique identifier.
Non-explicit definition: does not rely on traditional "team" settings, but on repository
access permissions.
```

```
2. Access and Authentication
Access condition: Any member with access to the repository.
Identity verification: OAuth authentication to obtain tokens, enabling read/write of shared memories.

3. Sync Mechanism (Pull-Push Mode)
The system ensures consistency across devices through the following flow:
Session start: Pull operation fetches the latest version from the server and overwrites local data.
Session end: Push operation syncs local incremental changes back to the server.

4. Conflict Handling and Security
Conflict management: ETag optimistic locking effectively resolves data conflicts during concurrent edits.
Privacy protection: File scanning is performed before upload. If API keys, tokens, or other sensitive credentials are detected, the file is automatically skipped, preventing sensitive information leakage.

5. Technical Features
Independence: Does not consume Git version history, does not rely on local file system sharing.
Cloud-driven: Purely relies on API and OAuth Token for cross-device sync.
```

---

5. Core Data Structures

5.1 MemoryHeader (Scan Result)

```typescript
// src/memdir/memoryScan.ts
export type MemoryHeader = {
  filename: string           // filename (without path)
  filePath: string          // absolute path
  mtimeMs: number           // file modification time (milliseconds Unix epoch)
  description: string | null // description in frontmatter (used for manifest)
  type: MemoryType | undefined // type in frontmatter (may be missing)
}
```

Purpose: Return type of `scanMemoryFiles()`, used for manifest formatting and **relevance retrieval**. **Only reads the first 30 lines of frontmatter, not full content**.

5.2 RelevantMemory (Retrieval Result)

```typescript
// src/memdir/findRelevantMemories.ts
export type RelevantMemory = {
  path: string    // absolute path to memory file
  mtimeMs: number  // modification time (used for freshness calculation)
}
```

See 6. Retrieval and Search Algorithm for the retrieval process details.

5.3 MemoryFileInfo (Complete Memory File)

```ts
// src/utils/claudemd.ts
export type MemoryFileInfo = {
  path: string                        // absolute file path
  type: MemoryType                    // 'Managed' | 'User' | 'Project' | 'Local'
                                       // 'AutoMem' | 'TeamMem'
  content: string                     // parsed plain text content
  parent?: string                      // parent file path containing this file (for
@include tracking)
  globs?: string[]                    // paths field in frontmatter (conditional rules)
  contentDiffersFromDisk?: boolean    // whether in-memory content is out of sync with disk
  rawContent?: string                 // raw Markdown content (with frontmatter)
}
```

5.4 SyncState (Team Memory Sync State)

```ts
// src/services/teamMemorySync/index.ts
export type SyncState = {
  lastKnownChecksum: string | null     // ETag, for HTTP conditional requests (If-None-
Match); used for next pull's If-None-Match (whole-repo level check for changes)

  serverChecksums: Map<string, string> // key (relative path) -> SHA-256 hash; used for this
push's delta calculation (file-level check for what changed)

  serverMaxEntries: number | null      // max entry count limit returned by server (learned
from 413 responses); used for pre-push local truncation (avoid triggering 413)
}
```

Note

See 9. Team Memory Cloud Sync System

- For syncing the latest team memory on the server
- Team memory exists on the server, essentially a shared memory database. Each member's local is just a cached copy — pull syncs the latest, push merges their changes up

```
The server computes a SHA-256 hash digest of the entire team memory snapshot.
Purpose is conditional requests:

Local pull:
  Request header with If-None-Match: "<previous checksum>"
  Server compares:
    checksum unchanged → returns 304 Not Modified (no change, download nothing)
    checksum changed → returns full data + new checksum

This way, each pull doesn't need to download full content — only transfer data when
something actually changed
```

5.5 SyncResult Various Response Types

```ts
// src/services/teamMemorySync/types.ts
export type TeamMemorySyncFetchResult = {
```

```
  success: boolean
  data?: TeamMemoryData
  isEmpty?: boolean
  notModified?: boolean        // HTTP 304, equivalent to last response
  checksum?: string            // ETag
  error?: string
  skipRetry?: boolean          // permanent failure, skip retry
  errorType?: 'auth' | 'timeout' | 'network' | 'parse' | 'unknown'
  httpStatus?: number
}

export type TeamMemorySyncPushResult = {
  success: boolean
  filesUploaded: number
  checksum?: string
  conflict?: boolean          // HTTP 412, need to refresh hash and retry
  error?: string
  skippedSecrets?: SkippedSecretFile[] // skipped files with sensitive info
  errorType?: 'auth' | 'timeout' | 'network' | 'conflict' |
              'unknown' | 'no_oauth' | 'no_repo'
  httpStatus?: number
}
```

Note

- This is a file sync service for the team-shared knowledge base. The service is responsible for uploading local files to the server (push) and fetching from the server (pull).
- **Sensitive data is sanitized before uploading memories**, see 9.6 Secret Scanner for details.

5.6 SessionMemoryConfig (Session Memory Configuration)

```
// src/services/SessionMemory/sessionMemoryUtils.ts
export type SessionMemoryConfig = {
  minimumMessageTokensToInit: number      // minimum tokens to start initialization
  minimumTokensBetweenUpdate: number     // minimum incremental tokens between updates
  toolCallsBetweenUpdates: number        // minimum tool calls between updates
}

export const DEFAULT_SESSION_MEMORY_CONFIG: SessionMemoryConfig = {
  minimumMessageTokensToInit: 10000,
  minimumTokensBetweenUpdate: 5000,
  toolCallsBetweenUpdates: 3,
}
```

---

6. Retrieval and Search Algorithm

6.1 Memory File Scanning (scanMemoryFiles)

File: `src/memdir/memoryScan.ts`

```
export async function scanMemoryFiles(
  memoryDir: string,
  signal: AbortSignal,
): Promise<MemoryHeader[]>
```

Algorithm steps:

1. Call `readdir(memoryDir, { recursive: true })` to traverse the directory
2. Filter all `.md` files (exclude MEMORY.md itself)
3. For each file:
   - Call `readFileInRange(filePath, { maxLines: 30 })` to read only the first 30 lines -> corresponding to frontmatter
   - Call `parseFrontmatter(raw)` to parse frontmatter
   - `readFileInRange` internally returns `mtimeMs` simultaneously (no separate stat needed)
4. Extract `{ filename, filePath, mtimeMs, description, type }`
5. Sort by `mtimeMs` descending (newest first)
6. Truncate to at most 200 files (`MAX_MEMORY_FILES`)
7. Return `MemoryHeader[]`

> **Single-pass read-then-sort optimization**: Source code comment (`memoryScan.ts`) explains this design:
>
> ```
> // Single-pass: readFileInRange stats internally and returns mtimeMs, so we
> // read-then-sort rather than stat-sort-read. For the common case (N ≤ 200)
> // this halves syscalls vs a separate stat round; for large N we read a few
> // extra small files but still avoid the double-stat on the surviving 200.
> ```

Manifest formatting:

```
export function formatMemoryManifest(memories: MemoryHeader[]): string {
  return memories.map(m => {
    const tag = m.type ? `[${m.type}] ` : ''
    const ts = new Date(m.mtimeMs).toISOString()
    return m.description
      ? `- ${tag}${m.filename} (${ts}): ${m.description}`
      : `- ${tag}${m.filename} (${ts})`
  }).join('\n')
}
```

Example output:

```
- [user] user_role.md (2026-03-30T10:00:00.000Z): User is a data scientist focused on
observability
- [feedback] feedback_testing.md (2026-03-29T08:00:00.000Z)
- [project] project_ingest.md (2026-03-28T15:30:00.000Z): Linear project INGEST tracks
pipeline bugs
```

6.2 AI Relevance Retrieval (findRelevantMemories)

File: `src/memdir/findRelevantMemories.ts`

This is the core intelligence layer of the memory system, using the Sonnet model for side queries.

```
export async function findRelevantMemories(
  query: string,                    // current user message/question
  memoryDir: string,                 // memory directory
  signal: AbortSignal,
  recentTools: readonly string[] = [],  // recently used tools (context signal)
  alreadySurfaced: ReadonlySet<string> = new Set(), // already surfaced memories (dedup)
): Promise<RelevantMemory[]>
```

Algorithm steps:

```
Step 1: Call scanMemoryFiles() to get manifest of all memories
Step 2: Format manifest (with timestamps and descriptions)
Step 3: Build System Prompt (SELECT_MEMORIES_SYSTEM_PROMPT):
        "You are a memory retrieval assistant. Given a user query and a list of
         available memories, select up to 5 memories most relevant to the query.
         Prefer memories with recent timestamps and direct topic matches.
         Output JSON: { "selected_memories": ["filename1", "filename2", ...] }
         Exclude: memories already surfaced in this conversation."

Step 4: Send request using Sonnet model (sideQuery)
Step 5: Parse JSON response, extract selected_memories array
Step 6: Filter: exclude memories already surfaced in alreadySurfaced
Step 7: Return RelevantMemory[] (path + mtimeMs)
```

Key design: Side Query means relevance retrieval occurs in an independent call, consuming no main conversation tokens and not affecting the main reasoning flow.

6.3 Conditional Rules

`claudemd.ts` supports the `paths` field in frontmatter for conditional memories:

```
---
name: vscode-settings-hint
description: VSCode special settings hint
type: user
paths:
  - "**/.vscode/**"
  - "**/settings.json"
---

When the user edits VSCode configuration...
```

The system matches the current working directory path via `glob` patterns, injecting the memory only when matched.

> **Note**
>
> This is a typical **context-aware** or **dynamic prompt injection** design. Simply put:
>
> - Normal memories are "globally effective" — once loaded, they always exist in context

- Conditional rules allow memory files to declare: **"this memory is only useful when I'm editing specific paths,"** thus injected only on match
  Personal speculation: this is a powerful feature because binding to paths enables:
- Different projects, file types, and work phases automatically switch different behavioral norms and prompts, without manual user specification
- Teams can declare project constitutions in team memory, and each member's Claude Code automatically operates based on this constitution when entering the workspace
  Using lazy loading, this feature becomes an activation and loading trigger for memories. If the path is a skill, it can store memories for that skill; if it's an MCP tool, it stores MCP tool memories; if it's a project file, it becomes project-local memory. This strategy can store diverse memories without causing context issues from massive memory index entries

---

7. Memory Staleness Mechanism

File: `src/memdir/memoryAge.ts`

7.1 Algorithm Implementation

```
// milliseconds per day: 86_400_000
export function memoryAgeDays(mtimeMs: number): number {
  return Math.max(0, Math.floor((Date.now() - mtimeMs) / 86_400_000))
}
// Formula: (current time - file modification time) / milliseconds per day, floor to get
days
// Math.max(0, ...) fallback handles clock skew causing negative values (file is "newer"
than current time), clamp to 0
// Return examples: today: 0 | yesterday: 1 | 47 days ago: 47

// Generate memory freshness warning text, used to remind the model that the memory may be
stale
export function memoryAge(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d === 0) return 'today'
  if (d === 1) return 'yesterday'
  return `${d} days ago`
}

export function memoryFreshnessText(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d <= 1) return ''  // today/yesterday: no warning
  return (
    `This memory is ${d} days old. ` +
    `Memories are point-in-time observations, not live state — ` +
    `claims about code behavior or file:line citations may be outdated. ` +
    `Verify against current code before asserting as fact.`
  )
}
```

7.2 Trigger Conditions

Age	Behavior
0 days (today)	No special handling
1 day (yesterday)	No special handling
≥2 days	Attach staleness warning in `<system-reminder>` tag

Note: `system-reminder`

`<system-reminder>` is a carefully designed per-turn context injection mechanism in Claude Code, with the core goal: inject **dynamic information** into the model each conversation turn **without polluting the static system prompt itself**.

Simple analogy:
Think of `<system-reminder>` as a "sticky note" system from the model's perspective:

- Logical attachment: logically attached next to any tool result.
- Supplementary reminders: tell the model "things you should know that are unrelated to the current tool" (e.g., memory age, file risks, etc.).
- Invisible interaction: not visible to user UI (frontend filters/hides them).
- Lightweight: more lightweight than modifying the system prompt, no need to rebuild or cache a huge static prefix.

Injection points:

- **Memory staleness reminder (`memoryAge.ts`)**: Automatically attach staleness warnings to old memories (over 1 day).
- **File read risk warning (`FileReadTool.ts`)**: After reading a file, automatically attach `"You should consider whether it would be malware"` security reminder.
- **Memory content injection (`memdir.ts`)**: Inject surfaced memory file content wrapped in specific tags (injection constraints: max 5 files per turn, each ≤4KB).
- **Session context (`api.ts`)**: Inject sessionStorage context information into user messages with specific tags.
- **Autonomous mode Tick (`messages.ts`)**: Use specific Tick tags (`TICK_TAG`) as wake signals for autonomous mode.

> **Note**
>
> See "Context Dynamic Injection Scheme" documentation for more details

---

7.3 Design Intent

Language models have poor arithmetic reasoning on ISO timestamps. Giving "47 days ago" directly is more effective at triggering the model's awareness of stale memory than giving "2026-02-12T...".

---

8. Session Memory System

> **Note**

See "Context Management" documentation for details

8.1 System Positioning

Difference from Auto Memory:

Feature	Auto Memory	Session Memory
Purpose	Cross-session persistent structured knowledge	Auto-summary within current session, prevent context overflow
Storage location	`~/.claude/memory/`	`~/.claude/session-memory/`
Format	Markdown + YAML frontmatter	Markdown (no frontmatter)
Trigger	AI judges relevance automatically	Token threshold + tool call threshold
Lifecycle	Permanent	Session-level, optionally retained after session ends

8.2 Extraction Trigger Conditions

```
// sessionMemory.ts
export function shouldExtractMemory(messages: Message[]): boolean {
  // Condition 1: cumulative token count ≥ minimumMessageTokensToInit (default 10,000)
  // Condition 2: growth since last extraction ≥ minimumTokensBetweenUpdate (default 5,000)
  // Condition 3: at least toolCallsBetweenUpdates tool calls since last extraction (default
3)
  // Condition 4: current is a natural pause point in conversation (last assistant message
has no tool call)
  // —— all must be met to trigger extraction
}
```

8.3 Session Memory Template Structure

`src/services/SessionMemory/prompts.ts` — `DEFAULT_SESSION_MEMORY_TEMPLATE`:

```
# Session Title
## Current State
## Task specification
## Files and Functions
## Workflow
## Errors & Corrections
## Codebase and System Documentation
## Learnings
## Key results
## Worklog
```

The system dynamically calculates token size for each section. If total length exceeds `MAX_TOTAL_SESSION_MEMORY_TOKENS` (12,000), it trims by priority (worklog > code documentation > intermediate steps).

8.4 Extraction Execution Flow

```
// sessionMemoryUtils.ts
// Global state (session-level)
```

```
let lastSummarizedMessageId: string | undefined
let extractionStartedAt: number | undefined
let tokensAtLastExtraction = 0
let sessionMemoryInitialized = false

// Main extraction function (sequential, prevents concurrency)
const extractSessionMemory = sequential(async function (context) {
  // 1. Get current session messages
  // 2. Check shouldExtractMemory() —— four conditions
  // 3. Call AI model with:
  //    - Current session-memory file content (if any)
  //    - Update prompt (incremental update, not full rewrite)
  //    - Template structure
  // 4. Write AI output to ~/.claude/session-memory/<sessionId>.md
  // 5. Update global state
})
```

8.5 Session Memory Compaction

File: `src/services/compact/sessionMemoryCompact.ts`

When context is about to overflow, the system uses summary information from session memory to decide which messages to retain:

```
// Priority (high to low):
// 1. File change records (git diff output)
// 2. Error messages
// 3. User explicit constraints/preferences
// 4. Decision records
// —— Lowest priority: intermediate reasoning steps, repetitive operation patterns

export function calculateMessagesToKeepIndex(
  messages: Message[],
  lastSummarizedIndex: number,
): number {
  // Returns starting index of messages to keep
  // Auto-adjusts: ensures not cutting off tool_use / tool_result pairs
}
```

Compaction trigger threshold (configurable):

```
export const DEFAULT_SM_COMPACT_CONFIG = {
  minTokens: 10_000,           // minimum message token count
  minTextBlockMessages: 5,     // minimum text message count
  maxTokens: 40_000,          // max tokens after compaction
}
```

---

9. Team Memory Cloud Sync System

9.1 API Endpoints

Method	Endpoint	Purpose
GET	`/api/claude_code/team_memory?repo={owner/repo}`	Pull team memory
GET	`/api/claude_code/team_memory?repo={owner/repo}&view=hashes`	Get hash metadata only (for delta calculation)
PUT	`/api/claude_code/team_memory?repo={owner/repo}`	Batch upload/update memories

9.2 Sync State Machine

```
export type SyncState = {
  lastKnownChecksum: string | null   // ETag (for If-None-Match conditional request), if server content unchanged, returns 304, skip writing to local (saving I/O)
  serverChecksums: Map<string, string> // key -> SHA-256(hex), before each local push, compare this Map — only upload files whose hash changed, skip others
  serverMaxEntries: number | null     // server-set entry limit. Local remembers this number to avoid next push being rejected by 413
}
```

9.3 Full Sync Algorithm

`syncTeamMemory()` = `pullTeamMemory()` + incremental `pushTeamMemory()`

Pull flow:

```
1. Check if syncState.serverChecksums is empty
   → Empty: full pull (first sync)
   → Non-empty: conditional request with If-None-Match: <checksum>
2. HTTP 200: write to local files, return filesWritten
3. HTTP 304 (notModified): no action needed
4. HTTP 413 (too_many_entries): extract max_entries from response, update serverMaxEntries
5. Other errors: return error + errorType
```

Push flow (incremental delta):

```
1. Read all files in local team memory directory
2. Calculate SHA-256(content) for each file
3. Compare with serverChecksums Map (key -> hash):
   → Same hash: skip
   → Different hash or new file: add to delta
4. secretScanner.scanForSecrets() —— scan all delta files
   → Sensitive info found: skip file, add to skippedSecrets[]
5. batchDeltaByBytes() —— batch by MAX_PUT_BODY_BYTES (200KB)

> **Why 200KB limit?** Source code comment (`teamMemorySync/index.ts`) reveals the history:
> ```
> // #21969 removed the client entry-count cap; cold pushes from heavy users
> // then sent 300KB-1.4MB bodies and hit the gateway body-size cap.
> // 200KB leaves headroom under the observed threshold and keeps a single
> // -entry-at-MAX_FILE_SIZE_BYTES solo batch (~250KB) just under the real
> // gateway limit.
> ```
>
```

```
> Background: There was a client entry-count hard cap, but it was removed (#21969). After
that, heavy users started sending 300KB-1.4MB bodies, triggering the API Gateway's body-size
limit (~256-512KB, timeout ~750ms vs normal 2.3s).
>
> Solution: Set 200KB soft cap, batch on client side so that no single request exceeds the
gateway limit. `MAX_FILE_SIZE_BYTES = 250KB` is the per-file limit, so a single-file solo
batch (~250KB) would be just over 200KB but might pass (leaving headroom).

6. Send PUT request (with If-Match: <lastKnownChecksum>)
7. HTTP 200: update syncState (checksum + serverChecksums)
8. HTTP 412 (conflict): re-request view=hashes, refresh serverChecksums, recalculate delta,
retry (max 2 times)
```

9.4 Hash Algorithm

```python
// src/services/teamMemorySync/index.ts
export function hashContent(content: string): string {
  // Uses SHA-256, returns hex string
  // Used for:
  //   - Comparison between local files and server files
  //   - ETag (lastKnownChecksum) calculation
}
```

9.5 File System Watcher

`src/services/teamMemorySync/watcher.ts`:

```
Startup timing: on first write to a team memory file
Listening scope: team memory directory (recursive)
Debounce delay: DEBOUNCE_MS = 2000ms

Flow:
1. fs.watch({ recursive: true }, callback) listens for create/modify/change events
2. Any event triggers: reset debounceTimer (setTimeout 2000ms)
3. No new events within 2000ms: executePush()
4. If pushInProgress: set hasPendingChanges = true, wait for current push to complete before
pushing
5. Permanent failure (no_oauth/no_repo/4xx non-409/429): set pushSuppressedReason,
subsequent writes won't retry
```

9.6 Secret Scanner

`src/services/teamMemorySync/secretScanner.ts`:

Executes before upload, ensuring sensitive information never leaves the local machine.

```python
export function scanForSecrets(content: string): SecretMatch[]
// Returns: { ruleId: string, label: string }[]
// Does not log specific sensitive values, only records rule ID and label

// Scan rules (gitleaks subset, ~30+ rules):
// AWS_ACCESS_KEY, AWS_SECRET_KEY, GCP_API_KEY, AZURE_TOKEN,
// DIGITALOCEAN_TOKEN, ANTHROPIC_API_KEY, OPENAI_API_KEY,
```

```
// HUGGINGFACE_TOKEN, GITHUB_PAT, GITLAB_TOKEN,
// SLACK_TOKEN, TWILIO_TOKEN, SENDGRID_KEY,
// NPM_TOKEN, PYPI_TOKEN, DATABRICKS_TOKEN,
// GRAFANA_TOKEN, SENTRY_DSN, STRIPE_KEY,
// SHOPIFY_SECRET, SSH_PRIVATE_KEY, PGP_PRIVATE_KEY,
// MONGODB_URI, POSTGRESQL_STRING, ...
```

Note

Rule matching does not call a large model; uses precompiled regex (`compiledRules`), avoiding recompilation on each scan

---

10. Agent Memory System

Note

Agent Memory is a **memory between a specific named Agent and its tasks** (temporarily created sub-agents do not need or have long-term memory configured)
Built-in named agents (persistent agent instances created via /agent create):

- Explore — specialized code exploration, read-only, no file modification
- Plan — plan-making, plan first before execution
- General Purpose — default all-purpose agent
- Verification — verification agent (run tests, etc.)
- Statusline Setup — status bar configuration
- Code Guide — embedded Claude Code usage guide

10.1 Three Scopes

```
// src/tools/AgentTool/agentMemory.ts
export type AgentMemoryScope = 'user' | 'project' | 'local'
```

Scope	Path	Purpose	Core Design Intent
`user`	`~/.claude/agent-memory/<agentType>/`	Global private, shared across multiple projects	Knowledge applicable across all my projects
`project`	`<cwd>/.claude/agent-memory/<agentType>/`	Shared within project, committed to git, shared with team members	Knowledge shared between me and the team on this project
`local`	`<cwd>/.claude/agent-memory-local/<agentType>/`	Valid only locally without mount; with mount, syncs local memory to remote mount point via fixed path, shared with team members	Knowledge that belongs only to this machine/my personal knowledge (machine-specific things)

10.2 Snapshot System

File: `src/tools/AgentTool/agentMemorySnapshot.ts`

The snapshot mechanism syncs memory updates from external sources (e.g., remote configuration system) without destroying existing memories:

```
// Snapshot directory structure
<cwd>/.claude/agent-memory-snapshots/<agentType>/
└── snapshot.json           # Snapshot metadata (containing updatedAt)

// Sync state tracking
<cwd>/.claude/agent-memory/<agentType>/
└── .snapshot-synced.json   # Records last synced snapshot timestamp

export async function checkAgentMemorySnapshot(
  agentType: string,
  scope: AgentMemoryScope,
): Promise<{
  action: 'none' | 'initialize' | 'prompt-update'
  snapshotTimestamp?: string
}>
```

Decision logic:

- `none`: No snapshot or already synced, no action
- `initialize`: Local directory is empty, initialize from snapshot
- `prompt-update`: Snapshot is newer than synced version, prompt user to confirm whether to update

---

11. @include Directive in Detail

Note

- Memory content can @include a file; when reading memory, it expands the full content
- More intuitively: @include is like `import` in memory files, allowing one memory to reference another, supporting nesting with cycle and depth protection

11.1 Syntax

```
@path                    # Resolved from memory base path (~/.claude)
@./relative/path         # Relative to current file's directory
@~/home/absolute/path    # Expands ~ to user home directory
@/absolute/path          # Absolute path
```

11.2 Parsing Flow (claudemd.ts)

```
// MAX_INCLUDE_DEPTH = 5 (prevents circular reference explosion)

export async function processMemoryFile(
  filePath: string,
  type: MemoryType,
  processedPaths: Set<string>,  // Tracks processed paths, prevents cycles
  includeExternal: boolean,
  depth: number = 0,           // Recursion depth counter
```

```
  parent?: string,             // Parent file path (used for parent field)
): Promise<MemoryFileInfo[]> {
  if (depth > MAX_INCLUDE_DEPTH) return []
  if (processedPaths.has(filePath)) return []  // Cycle detection
  processedPaths.add(filePath)

  // 1. Read file
  // 2. Parse frontmatter
  // 3. Extract paths from @include directives
  // 4. Recursively process each included file
  // 5. Return all MemoryFileInfo (with parent field)
}
```

11.3 Supported File Types

The system maintains a `TEXT_FILE_EXTENSIONS` set (100+ extensions), including: `.md`, `.txt`, `.json`, `.yaml`, `.yml`, `.toml`, `.xml`, `.html`, `.css`, `.js`, `.ts`, `.py`, `.rb`, `.go`, `.rs`, `.java`, `.c`, `.cpp`, `.h`, `.sh`, `.bash`, `.zsh`, `.ps1`, `.sql`, `.graphql`, `.proto`, `.env`, `.ini`, `.cfg`, `.conf`, etc.

11.4 Conditional @include (paths field)

The `paths` field in frontmatter supports glob patterns:

```
---
name: docker-hints
description: Docker-related development hints
type: reference
paths:
  - "**/Dockerfile"
  - "**/docker-compose.yml"
  - "**/.dockerignore"
---

Inject this memory when the user is handling Docker-related files...
```

The system uses `resolveExcludePatterns()` to check if the current working directory path matches any pattern.

---

12. Memory Loading Order and Priority

`src/utils/claudemd.ts` — `getMemoryFiles()`:

12.1 Four-Level Loading Order

```
Priority 1 (lowest) → Managed (global directives)
  Path: /etc/claude-code/CLAUDE.md
  Description: System-level global directives, applicable to all users

Priority 2 → User (user global directives)
  Path: ~/.claude/CLAUDE.md
  Description: User's private global directives

Priority 3 → Project (project directives)
```

```
  Paths:
    <cwd>/CLAUDE.md
    <cwd>/.claude/CLAUDE.md
    <cwd>/.claude/rules/*.md
  Description: Can be committed to version control, shared within project

Priority 4 (highest) → Local (local private)
  Paths:
    <cwd>/CLAUDE.local.md
    <cwd>/.claude/rules.local/*.md
  Description: Should not be committed to version control (usually in .gitignore)
```

"Closer to CWD Wins" principle:
Multiple files of the same type: later-loaded overrides earlier-loaded. For example: if both `CLAUDE.md` and `.claude/CLAUDE.md` exist in the project directory, the latter has higher priority.

12.2 Content Merge Strategy

The system uses **override merge**, not concatenation:

- Same key (e.g., same section name): latter content overrides former
- Different keys: both retained
- Final output is a single string

> **Note**
>
> Override merge solves **which one wins when multiple memory file contents conflict**, not preventing duplicate saves of the same memory

---

13. Security Mechanisms

13.1 Path Traversal Protection

`src/memdir/teamMemPaths.ts` — `validateTeamMemWritePath()`:

```
async function validateTeamMemWritePath(filePath: string): Promise<string> {
  // 1. Check null bytes (\0): inserting \0 in a path, many language low-level APIs treat it as string terminator, causing actual path truncation
  // 2. Normalize path (resolve . and ..)
  // 3. realpathDeepestExisting() —— resolve to nearest existing ancestor directory
  //    - Prevents directory traversal via symbolic links
  // 4. Verify normalized path is still within team memory directory
  // 5. Reject Unicode normalization attacks (NFKC/NFKC_CaseFold) — visually identical but different character codes, bypassing string checks: server NFKC-normalizes, then the two become different characters — but if check logic runs before normalization, it could be bypassed
  //    - "．．" (U+2024) vs ".." (U+002E U+002E)
  // 6. Reject URL-encoded path traversal (%2e%2e/%2f): %2e%2e%2f decodes to ../, some validation logic only checks strings without first decoding URLs
}
```

> **Note**

- This is **security validation to prevent path traversal attacks**: when uploading files, verify the path won't escape the team memory directory — blocking various attack vectors: .. traversal, symlink traversal, Unicode confusion, URL encoding bypass
- **Path traversal attack**: If an attacker can write a path as ../.ssh/authorized_keys, they could overwrite /home/user/.ssh/authorized_keys — writing files **outside** the team memory directory

13.2 Configuration Injection Protection

`src/memdir/paths.ts`:

- `autoMemoryDirectory` is **prohibited** from being set in project-level `settings.json`
- Can only be injected from `policy`, `local`, `user` three safe levels
- Reason: project settings can be controlled by any repository contributor, potentially pointing to `~/.ssh/` or other sensitive directories

13.3 Sensitive Information Protection

See 9.6 Secret Scanner: Client-side scanning ensures sensitive information never leaves the local machine.

13.4 File Permissions

- Memory directory: `0o700` (owner read/write/execute only)
- Memory files: `0o600` (owner read/write only)

---

14. Complete Index of Key Functions

14.1 Auto Memory Module (memdir/)

File	Function Signature	Purpose
`memoryTypes.ts`	`parseMemoryType(raw: unknown)`	Parse frontmatter type value
`memoryTypes.ts`	`TYPES_SECTION_COMBINED` (constant)	Type description prompt with scope tags
`memoryTypes.ts`	`TYPES_SECTION_INDIVIDUAL` (constant)	Type description prompt without scope tags
`memoryScan.ts`	`scanMemoryFiles(memoryDir, signal)`	Recursive scan, returns MemoryHeader[]
`memoryScan.ts`	`formatMemoryManifest(memories)`	Format as standard manifest text
`memoryAge.ts`	`memoryAgeDays(mtimeMs)`	Calculate age (days, floor)
`memoryAge.ts`	`memoryAge(mtimeMs)`	Human-readable age ("today"/"N days ago")
`memoryAge.ts`	`memoryFreshnessText(mtimeMs)`	Staleness warning text (triggers ≥2 days)

File	Function Signature	Purpose
`memoryAge.ts`	`memoryFreshnessNote(mtimeMs)`	Staleness warning (with `<system-reminder>` tag)
`memdir.ts`	`truncateEntrypointContent(raw)`	Dual-limit truncation (lines + bytes)
`memdir.ts`	`buildMemoryLines(displayName, memoryDir, extraGuidelines?, skipIndex?)`	Build type memory description (without MEMORY.md content)
`memdir.ts`	`buildMemoryPrompt(params)`	Build complete prompt with MEMORY.md content
`memdir.ts`	`loadMemoryPrompt()`	Async load and assemble complete memory prompt
`memdir.ts`	`ensureMemoryDirExists(memoryDir)`	Ensure directory exists (with guidance text)
`paths.ts`	`isAutoMemoryEnabled()`	Check global auto memory toggle
`paths.ts`	`isExtractModeActive()`	Check if extraction mode (background agent) is active
`paths.ts`	`getMemoryBaseDir()`	Get memory base directory (default ~/.claude)
`paths.ts`	`getAutoMemPath()`	Resolve auto memory directory (with memoize cache)
`paths.ts`	`getAutoMemDailyLogPath(date?)`	Get daily log path for specified date
`paths.ts`	`getAutoMemEntrypoint()`	Get MEMORY.md path
`paths.ts`	`isAutoMemPath(absolutePath)`	Check if path is within auto memory directory
`paths.ts`	`hasAutoMemPathOverride()`	Check for env var override
`teamMemPaths.ts`	`PathTraversalError` (class)	Path traversal detection exception
`teamMemPaths.ts`	`isTeamMemoryEnabled()`	Check if team memory is enabled
`teamMemPaths.ts`	`getTeamMemPath()`	Get team memory directory
`teamMemPaths.ts`	`getTeamMemEntrypoint()`	Get team MEMORY.md

File	Function Signature	Purpose
`teamMemPaths.ts`	`validateTeamMemWritePath(absPath)`	Validate write path safety (async)
`teamMemPaths.ts`	`validateTeamMemKey(relativeKey)`	Validate server-returned relative path key
`teamMemPaths.ts`	`isTeamMemFile(filePath)`	Check if path is in team memory directory
`teamMemPrompts.ts`	`buildCombinedMemoryPrompt(extraGuidelines?, skipIndex?)`	Combine private + team memory prompt
`findRelevantMemories.ts`	`findRelevantMemories(query, memoryDir, signal, recentTools?, alreadySurfaced?)`	AI relevance retrieval main entry
`findRelevantMemories.ts`	`selectRelevantMemories(query, memories, signal, recentTools)`	Call Sonnet sideQuery to select memories

14.2 Session Memory Module (services/SessionMemory/)

File	Function/Constant	Purpose
`sessionMemory.ts`	`shouldExtractMemory(messages)`	Four-condition check for extraction trigger
`sessionMemory.ts`	`initSessionMemory()`	Initialize session memory
`sessionMemory.ts`	`manuallyExtractSessionMemory(messages, toolUseContext)`	Manually trigger extraction
`sessionMemory.ts`	`createMemoryFileCanUseTool(memoryPath)`	Build tool permission function (restrict writing to memory files)
`sessionMemory.ts`	`resetLastMemoryMessageUuid()`	Reset last memory message ID
`sessionMemoryUtils.ts`	`getLastSummarizedMessageId()`	Get last summarized message ID
`sessionMemoryUtils.ts`	`markExtractionStarted()` / `markExtractionCompleted()`	Mark extraction state
`sessionMemoryUtils.ts`	`waitForSessionMemoryExtraction()`	Wait for extraction to complete (max 15000ms)
`sessionMemoryUtils.ts`	`getSessionMemoryContent()`	Read session memory file content
`sessionMemoryUtils.ts`	`setSessionMemoryConfig(config)`	Dynamic config update (GrowthBook remote config)
`sessionMemoryUtils.ts`	`hasMetInitializationThreshold(tokenCount)`	Check initialization threshold
`sessionMemoryUtils.ts`	`hasMetUpdateThreshold(tokenCount)`	Check incremental update threshold

File	Function/Constant	Purpose
`sessionMemoryUtils.ts`	`resetSessionMemoryState()`	Reset all global state (at session end)
`prompts.ts`	`loadSessionMemoryTemplate()`	Load template (supports file override)
`prompts.ts`	`loadSessionMemoryPrompt()`	Load extraction system prompt
`prompts.ts`	`buildSessionMemoryUpdatePrompt(currentNotes, notesPath)`	Build incremental update prompt
`prompts.ts`	`isSessionMemoryEmpty(content)`	Check if session memory is empty
`prompts.ts`	`truncateSessionMemoryForCompact(content)`	Truncate session memory during compaction

14.3 Team Memory Sync Module (services/teamMemorySync/)

File	Function/Constant	Purpose
`index.ts`	`createSyncState()`	Create initial sync state
`index.ts`	`hashContent(content)`	SHA-256 hash calculation
`index.ts`	`batchDeltaByBytes(delta)`	Batch by 200KB
`index.ts`	`isTeamMemorySyncAvailable()`	Check OAuth + repo availability
`index.ts`	`pullTeamMemory(state, options?)`	Pull (supports first-time full/conditional request)
`index.ts`	`pushTeamMemory(state)`	Push (includes secret scan + delta calculation)
`index.ts`	`syncTeamMemory(state)`	Full sync (pull + push)
`watcher.ts`	`startTeamMemoryWatcher()`	Start fs.watch + debounced push
`watcher.ts`	`notifyTeamMemoryWrite()`	Trigger debounced push
`watcher.ts`	`stopTeamMemoryWatcher()`	Stop watcher
`watcher.ts`	`isPermanentFailure(result)`	Determine if permanent failure (non-409/429 4xx)
`secretScanner.ts`	`scanForSecrets(content)`	Detect sensitive info (returns ruleId + label)
`secretScanner.ts`	`getSecretLabel(ruleId)`	Get rule label (human-readable name)
`secretScanner.ts`	`redactSecrets(content)`	Sanitize (replace with `[REDACTED]`)
`types.ts`	`TeamMemoryDataSchema` (Zod)	Server response JSON schema
`types.ts`	`TeamMemoryTooManyEntriesSchema` (Zod)	413 error schema

14.4 Agent Memory Module (tools/AgentTool/)

File	Function	Purpose
`agentMemory.ts`	`getAgentMemoryDir(agentType, scope)`	Get agent memory directory
`agentMemory.ts`	`getAgentMemoryEntrypoint(agentType, scope)`	Get agent memory MEMORY.md
`agentMemory.ts`	`loadAgentMemoryPrompt(agentType, scope)`	Load agent memory content

File	Function	Purpose
`agentMemory.ts`	`isAgentMemoryPath(absolutePath)`	Check if path is in agent memory directory
`agentMemorySnapshot.ts`	`checkAgentMemorySnapshot(agentType, scope)`	Check snapshot state, decide action
`agentMemorySnapshot.ts`	`initializeFromSnapshot(agentType, scope, timestamp)`	Initialize local memory from snapshot
`agentMemorySnapshot.ts`	`replaceFromSnapshot(agentType, scope, timestamp)`	Replace local memory with snapshot
`agentMemorySnapshot.ts`	`markSnapshotSynced(agentType, scope, timestamp)`	Mark snapshot as synced

14.5 Core Utilities Module (utils/)

File	Function/Constant	Purpose
`claudemd.ts`	`processMemoryFile(filePath, type, processedPaths, includeExternal, depth?, parent?)`	Parse single memory file (with @include recursion)
`claudemd.ts`	`processMdRules({ rulesDir, type, processedPaths, includeExternal, conditionalRule? })`	Process rules directory
`claudemd.ts`	`getMemoryFiles(forceIncludeExternal?)`	Main entry: get all memory files (with memoize)
`claudemd.ts`	`resetGetMemoryFilesCache(reason?)`	Reset memory files cache
`claudemd.ts`	`getClaudeMds(memoryFiles, filter?)`	Filter and concatenate to plain text
`claudemd.ts`	`getManagedAndUserConditionalRules(targetPath, processedPaths)`	Get conditional rules
`claudemd.ts`	`getExternalClaudeMdIncludes(files)`	Get external @include file list
`claudemd.ts`	`stripHtmlComments(content)`	Remove HTML comments
`claudemd.ts`	`MAX_MEMORY_CHARACTER_COUNT = 40000`	Max memory character count
`claudemd.ts`	`MAX_INCLUDE_DEPTH = 5`	@include max nesting depth
`memoryFileDetection.ts`	`detectSessionFileType(filePath)`	Detect session memory/transcript file
`memoryFileDetection.ts`	`isAutoMemFile(filePath)`	Detect auto memory file
`memoryFileDetection.ts`	`memoryScopeForPath(filePath)`	Determine personal/team scope
`memoryFileDetection.ts`	`isAutoManagedMemoryFile(filePath)`	Detect Claude-managed memory file

File	Function/Constant	Purpose
`memoryFileDetection.ts`	`isShellCommandTargetingMemory(command)`	Detect if shell command targets memory files

14.6 Auto-Dream Module (services/autoDream/)

File	Function/Constant	Purpose
`autoDream.ts`	`initAutoDream()`	Module initialization (create runner closure)
`autoDream.ts`	`executeAutoDream(context, appendSystemMessage?)`	Per-turn stop hook entry (non-blocking)
`autoDream.ts`	`isGateOpen()`	Check time/mode/flag three gates
`autoDream.ts`	`getConfig()`	Get GrowthBook config (tengu_onyx_plover)
`autoDream.ts`	`makeDreamProgressWatcher(taskId, setAppState)`	Listen to forked agent messages, collect file writes
`consolidationLock.ts`	`readLastConsolidatedAt()`	Read lock file mtime (i.e., lastConsolidatedAt)
`consolidationLock.ts`	`tryAcquireConsolidationLock()`	Atomically acquire lock, return priorMtime or null
`consolidationLock.ts`	`rollbackConsolidationLock(priorMtime)`	Rollback lock mtime (on fork failure)
`consolidationLock.ts`	`listSessionsTouchedSince(sinceMs)`	Get session ID list with mtime > sinceMs
`consolidationLock.ts`	`recordConsolidation()`	Record timestamp when manually triggered by /dream
`consolidationPrompt.ts`	`buildConsolidationPrompt(memoryRoot, transcriptDir, extra)`	Build four-phase consolidation prompt
`config.ts`	`isAutoDreamEnabled()`	Check flag status (settings > GrowthBook)

14.7 Memory Extraction Module (services/extractMemories/)

File	Function/Constant	Purpose
`extractMemories.ts`	`initExtractMemories()`	Module initialization (create extractor/drainer closures)
`extractMemories.ts`	`executeExtractMemories(context, appendSystemMessage?)`	Per-turn stop hook entry
`extractMemories.ts`	`drainPendingExtraction(timeoutMs?)`	Wait for all in-flight extractions before graceful shutdown
`extractMemories.ts`	`createAutoMemCanUseTool(memoryDir)`	Build tool permission function (shared with autoDream)
`extractMemories.ts`	`hasMemoryWritesSince(messages, sinceUuid)`	Detect if main agent has already written memories (mutex skip)
`extractMemories.ts`	`countModelVisibleMessagesSince(messages, sinceUuid)`	Count newly visible messages

File	Function/Constant	Purpose
`extractMemories.ts`	`extractWrittenPaths(agentMessages)`	Extract Edit/Write paths from agent output
`extractMemories.ts`	`isModelVisibleMessage(message)`	Check if message is visible to model (excludes progress/system/attachment)
`prompts.ts`	`buildExtractAutoOnlyPrompt(...)`	Build auto-only memory extraction prompt
`prompts.ts`	`buildExtractCombinedPrompt(...)`	Build private + team memory extraction prompt (with scope guidance)
`prompts.ts`	`opener(newMessageCount, existingMemories)`	Opening shared by both prompt variants

createAutoMemCanUseTool security design (`extractMemories.ts`):

```
export function createAutoMemCanUseTool(memoryDir: string): CanUseToolFn {
  return async (tool: Tool, input: Record<string, unknown>) => {
    // REPL mode: primitive tools are hidden, forked agent calls REPL.
    // Giving fork a different tool list breaks prompt cache sharing (tool list is the cache
key).
    if (tool.name === REPL_TOOL_NAME) {
      return { behavior: 'allow', updatedInput: input }
    }
    // Read tools: unrestricted
    if (tool.name === FILE_READ_TOOL_NAME || tool.name === GREP_TOOL_NAME || tool.name ===
GLOB_TOOL_NAME) {
      return { behavior: 'allow', updatedInput: input }
    }
    // Bash: read-only only
    if (tool.name === BASH_TOOL_NAME) {
      if (tool.isReadOnly(parsed.data)) {  // tool.isReadOnly is BashTool's built-in
judgment
        return { behavior: 'allow', updatedInput: input }
      }
      return denyAutoMemTool(tool, 'Only read-only shell commands...')
    }
    // Edit/Write: only files within memoryDir
    if ((tool.name === FILE_EDIT_TOOL_NAME || tool.name === FILE_WRITE_TOOL_NAME) &&
'file_path' in input) {
      if (typeof filePath === 'string' && isAutoMemPath(filePath)) {
        return { behavior: 'allow', updatedInput: input }
      }
    }
    return denyAutoMemTool(tool, 'only Read/Grep/Glob/read-only Bash/Edit-Write-in-memdir
allowed')
  }
}
```

Design要点：

- REPL mode special handling: ANT has REPL mode enabled by default, primitive tools are hidden from the direct tool list, and forked agents can only call them via REPL. To prevent REPL from bypassing restrictions, inner primitive tool calls re-enter the `canUseTool` check.
- Tool list as cache key (`CacheSafeParams`): Giving a forked agent a different tool list means it can't reuse the parent process's prompt cache, so REPL mode is specially handled rather than changing the tool list.
- `isAutoMemPath` is security-critical: only files with paths starting with `getAutoMemPath()` are allowed to be written, preventing memory writes from being injected into other directories.
- `tool.isReadOnly(parsed.data)` : Directly uses BashTool's built-in read-only judgment, no need to maintain a command whitelist.

> **Note**
>
> Primitive tools: Primitive tools are the lowest-level file/command operation tools. In REPL mode, while their names are hidden from the model (shown as Read/Write/Bash etc.), the underlying implementation and rendering code still need to be aware of their existence.
>
> ```
> FileReadTool      ← Read files
> FileWriteTool     ← Write files
> FileEditTool      ← Edit files
> GlobTool          ← Glob file matching
> GrepTool          ← Content search
> BashTool          ← Execute commands
> NotebookEditTool  ← Jupyter Notebook editing
> AgentTool         ← Launch Agent
> ```

---

15. System Architecture Overview

15.1 Architecture Main Diagram

```
┌──────────────────────────────────────────────────────────────────┐
│                        User Input                                 │
└──────────────────────────────┬──────────────────────────────────┘
                               │
                               ▼
┌──────────────────────────────────────────────────────────────────┐
│              shouldShowClaudeMdExternalIncludesWarning()         │
│              Whether to warn about external @include files        │
└──────────────────────────────┬──────────────────────────────────┘
                               │
                               ▼
┌──────────────────────────────────────────────────────────────────┐
│                  getMemoryFiles() (memoized)                      │
│           Load all memory files by priority (with @include recursion) │
│   Managed → User → Project → Local (later overrides earlier)      │
└──────────────────────────────┬──────────────────────────────────┘
                               │
            ┌──────────────────┼──────────────────┐
            ▼                  ▼                  ▼
   ┌──────────────┐    ┌──────────────┐    ┌──────────────────┐
```

```
│  CLAUDE.md   │   │ Auto Memory   │   │  Team Memory Sync  │
│  (4-level   │   │ (scan+search) │   │ (Watcher+Push)      │
│   loading)  │   └──────────────┘   └────────────────────┘
└────────────┘           │
                         │
          ┌──────────────┼──────────────┐
          ▼              ▼              ▼

┌──────────────────┐  ┌──────────────┐  ┌────────────────────┐
│ scanMemoryFiles() │  │AI Relevance│  │ sessionMemory.ts │
│  (file scanning)  │  │findRelevant│  │ (extraction      │
└──────────────────┘  │ Memories() │  │  trigger check)   │
                      └──────────────┘  └────────────────────┘
                            │                 │
                            ▼                 ▼
                      ┌──────────────┐  ┌────────────────────┐
                      │ Manifest  │  │ extractSessionMem │
                      │ (text)    │  │ (AI incremental  │
                      └──────────────┘  │  summary)        │
                            │          └────────────────────┘
                            ▼                 │
                 ┌──────────────────────────────────────┐
                 │        buildMemoryPrompt()            │
                 │  Assemble: type description + relevant │
                 │  memories + freshness annotations     │
                 └──────────────────────────────────────┘
                                  │
                                  ▼
                 ┌──────────────────────────────────────┐
                 │  loadMemoryPrompt() / getClaudeMds()  │
                 │  MEMORY.md truncation (200 lines/25KB) │
                 └──────────────────────────────────────┘
                                  │
                                  ▼
                 ┌──────────────────────────────────────┐
                 │     Main System Prompt (with Memory)  │
                 └──────────────────────────────────────┘
                                  │
                  ┌───────────────┼────────────────────┐
                  │               │                    │
                  ▼               ▼                    ▼

          ┌──────────────┐   ┌──────────────┐   ┌────────────────┐
          │Claude Sonnet │   │  Tool Calls   │   │  Agent Memory    │
          │  Model       │   │   (Tools)     │   │  (Snapshot)      │
          └──────────────┘   └──────────────┘   └────────────────┘
                                   │
                        ┌──────────┼──────────────┐
                        ▼          ▼              ▼

                 ┌──────────┐ ┌──────────┐   ┌────────────────┐
                 │Write files │ │ Git ops    │   │ Session Memory   │
                 │(Write/Edit │ │ (Bash)     │   │ Compaction       │
                 └──────────┘ └──────────┘   └────────────────┘
                        │                          │
                        └──────────────────────────┘
                                   ▼
```

```
                    ┌──────────────────────┐
                    │ Team Memory Watcher│
                    │  (fs.watch debounce 2s) │
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │  secretScanner     │
                    │   (pre-upload scan) │
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │ pushTeamMemory()   │
                    │ (delta push)       │
                    └──────────────────────┘
```

15.2 Background Process Diagram

```
========== Background Processes (independent from main conversation)============

              ┌──────────────────────────────────────────┐
              │          executeExtractMemories()        │
              │   stopHooks.ts → triggers at end of each │
              │   conversation turn                      │
              │   Independent forked agent, writes to ~/<memory>/ │
              │   Tool restrictions: read-only + write only within memoryDir │
              └──────────────────────────────────────────┘
                                  │
                                  │ Mutex check (skipped if main agent already wrote)
                                  ▼
              ┌──────────────────────────────────────────┐
              │          executeAutoDream()              │
              │   stopHooks.ts → check three gates at end │
              │   of each turn                           │
              │   Time gate ≥24h + Session gate ≥5 + Lock gate │
              │   Forked agent, four-phase consolidation prompt │
              │   Results written to DreamTask UI (footer pill) │
              └──────────────────────────────────────────┘
                                  │
                                  │
                        ┌─────────▼─────────┐
                        │  .consolidate-   │
                        │  lock file       │
                        │  (mtime = last- │
                        │   ConsolidatedAt)│
                        └──────────────────┘
```

---

16. Algorithm Complexity Analysis

16.1 Operation Complexity Table

Operation	Time Complexity	Space Complexity	Notes
`scanMemoryFiles()`	O(n log n)	O(n)	n=file count, mostly from sorting

Operation	Time Complexity	Space Complexity	Notes
`formatMemoryManifest()`	O(n)	O(n)	manifest string construction
`findRelevantMemories()`	O(n) + O(1) API	O(n) manifest	AI call is side query, not counted in main complexity
`syncTeamMemory()`	O(n) hash + O(k) push	O(1) incremental	k=changed file count, k≤n
`shouldExtractMemory()`	O(1)	O(1)	condition check only
`@include` parsing	O(d), d≤5	O(d) call stack	depth-limited, d≤5
`truncateEntrypointContent()`	O(L+B)	O(1)	L=lines, B=bytes
`calculateMessagesToKeepIndex()`	O(m)	O(1)	m=message count, linear scan
`processMemoryFile()`	O(f × d)	O(d)	f=included file count, d=depth≤5

---

17. Auto-Dream System (Automatic Memory Consolidation)

17.1 System Positioning

Auto-Dream is Claude Code's **background memory consolidation mechanism**, running a specialized forked subagent in the background to deeply organize accumulated session memories. It is not a user-initiated command, but a background process that automatically triggers when specific conditions are met.

Relationship with main agent memory writing:

- The main agent writes memories in the conversation based on system prompt instructions (per-session)
- extractMemories does lightweight supplemental extraction at the end of each turn (also forked agent)
- Auto-Dream is a periodic **deep consolidation**, responsible for **merging, deduplication, error correction, and index updating**

Three-gate trigger mechanism (Gate Order, cheapest checks first):

```
Trigger condition check order:

Gate 1 — Time gate (inside isGateOpen)
  → ≥ minHours since last consolidation (default 24h)
  → Checked every turn: one stat call

Gate 2 — Session gate (inside isGateOpen)
  → Number of sessions with new changes since last consolidation ≥ minSessions (default 5)
  → Scan throttling: skip if less than SESSION_SCAN_INTERVAL_MS (10min) since last scan

Gate 3 — Lock gate
  → .consolidate-lock file mtime = lastConsolidatedAt
  → PID validation + 60min expiration protection, prevents zombie processes
```

> **Note**
>
> **Gate checks are ordered by computational cost from low to high. The time gate costs only one stat check; if skipped, check the session gate; the session gate has slightly higher computation but has 10-minute throttling; the lock gate is heaviest but is already the last step before triggering**

17.2 Lock File Mechanism (consolidationLock.ts)

Lock file path: `~/.claude/projects/<sanitized-git-root>/memory/.consolidate-lock`

Design要点:

- Lock file is placed inside the memory directory, following git root; different worktrees won't interfere with each other
- Lock file body = PID, mtime = lastConsolidatedAt (so mtime serves double duty as timestamp)
- Holder expiration time (HOLDER_STALE_MS) = 60 minutes, prevents PID recycling and reuse causing permanent deadlock
- rollbackConsolidationLock: rollback mtime after fork failure, allowing the time gate to pass next time

```
// Atomic lock acquisition flow
tryAcquireConsolidationLock():
  1. Read lock file mtime + PID
  2. If mtime < 60min and PID alive → lock held, return null
  3. Write current PID (mtime = now)
  4. Re-read to verify (prevent race), return null if PID doesn't match
  5. Return priorMtime (for rollback on failure)
```

17.3 Four-Phase Consolidation Prompt (consolidationPrompt.ts)

The consolidation prompt guides the forked agent through four phases:

```
Phase 1 — Orient
  ls memory directory + read MEMORY.md to understand current index
  Avoid duplication: check existing topic files before deciding what to write

Phase 2 — Gather recent signal
  Check sources by priority:
    1. Daily logs (logs/YYYY/MM/YYYY-MM-DD.md) — daily append logs
    2. Stale memories (facts that contradict current code)
    3. Transcript search (grep JSONL, exact match only, not full read)
  Principle: don't exhaustively read transcript, only find things already suspected
important

Phase 3 — Consolidate
  For each piece worth remembering:
    → Merge into existing topic files (rather than creating near-duplicates)
    → Relative dates → absolute dates ("yesterday" → "2026-04-07")
    → Delete facts that have been disproven

Phase 4 — Prune and index
  Update MEMORY.md (≤200 lines, ≤~25KB):
    → Delete pointers to stale/incorrect/superseded memories
    → Trim overly long entries (>200 chars should be shortened, move details to topic files)
    → Add pointers to new important memories
    → Resolve contradictions (when two files disagree, fix the incorrect one)
```

17.4 Tool Constraints

Auto-Dream forked agent tool constraints (from `createAutoMemCanUseTool`):

- Allowed: Read / Grep / Glob (unrestricted)
- Allowed: Read-only Bash (`ls` , `find` , `grep` , `cat` , `stat` , `wc` , `head` , `tail` , etc.)
- Allowed: Edit / Write **only within memory directory**
- Denied: Write-capable Bash (e.g., `>` redirection, `|` piping to files), MCP tools, Agent tools

> **Note**
>
> **This constraint is identical to extractMemories, shared via `createAutoMemCanUseTool()`**

17.5 DreamTask UI Integration

The Auto-Dream forked agent runs in the background and is invisible to the user — DreamTask "visualizes" it to the footer pill and Shift+Down floating panel.

```
// DreamTaskState structure
{
  type: 'dream',
  phase: 'starting' | 'updating',  // switches from starting to updating after receiving
Edit/Write
  sessionsReviewing: number,        // number of sessions to review
  filesTouched: string[],           // observed file paths (from tool_use, incomplete)
  turns: DreamTurn[],               // assistant text responses, tool calls collapsed to
count
  abortController?: AbortController,
  priorMtime: number,              // used for rollback when killing
}

// kill() behavior:
// 1. abort abortController (abort forked agent)
// 2. status → killed
// 3. Rollback lock mtime (rollbackConsolidationLock)
```

> **Note**
>
> **filesTouched is incomplete — only captures Edit/Write tool_use, not writes mediated by bash, so it represents "at least these were changed," not "only these were changed"**

17.6 GrowthBook Configuration and Feature Flag

```
// autoDream.ts — getConfig() reads GrowthBook
' tengu_onyx_plover': {
  minHours: 24,       // default 24 hours
  minSessions: 5,     // default 5 sessions
}

// config.ts — isAutoDreamEnabled() priority
1. settings.json's autoDreamEnabled (user explicit setting)
2. GrowthBook 'tengu_onyx_plover'.enabled (experiment group)
3. Return false
```

17.7 Relationship with /dream Skill

The `/dream` skill (`skills/bundled/dream.ts`) is a manually triggered dream command, behind the `KAIROS` or `KAIROS_DREAM` feature flag:

- When manually triggered, calls `recordConsolidation()` to optimistically update the timestamp
- Also uses the same consolidation prompt
- Also a forked agent, but goes through the /dream skill path, bypassing autoDream's three-gate check

---

18. ExtractMemories System (End-of-Session Memory Extraction)

18.1 System Positioning

ExtractMemories is a lightweight memory extraction triggered at the end of each conversation turn (`stopHooks`), running via a **forked agent**. It is not a replacement for the main agent writing its own memories, but a complementary mechanism — automatically mining extractable information when the main agent has not actively written memories.

18.2 Forked Agent Mode

Extraction uses `runForkedAgent()` , a perfect fork of the main conversation:

- Same system prompt (includes complete memory-saving instructions)
- Shares parent's prompt cache
- Runs independently, does not write back to main transcript (`skipTranscript: true`)
- Max 5 turns (`maxTurns: 5`) to prevent verification rabbit-hole

Mutex mechanism (hasMemoryWritesSince):
If the main agent has already written to a memory file in this conversation turn (by checking if tool_use contains Edit/Write to autoMemPath), skip extraction and advance the cursor directly.

```
// Mutex: main agent already wrote → skip extraction
if (hasMemoryWritesSince(messages, lastMemoryMessageUuid)) {
  lastMemoryMessageUuid = lastMessage.uuid  // Advance cursor, don't process
  return
}
```

18.3 Closure State and Trailing Run

The extraction module creates closures via `initExtractMemories()` , capturing the following state:

```
let lastMemoryMessageUuid        // Cursor: UUID of the last processed message
let inProgress = false           // Whether an extraction is currently running
let turnsSinceLastExtraction = 0 // Throttle counter
let pendingContext               // Backlog of context during execution (for trailing run)
let inFlightExtractions = Set    // All pending extraction Promises
```

Trailing Run mechanism:

- If another extraction is triggered during the main extraction run → backlog to `pendingContext`
- After current extraction completes, check `pendingContext` and execute trailing run (skip throttle)
- Ensures new messages are not missed

> **Note**
>
> - "Closure state" means: after the Agent ends the main conversation, the background memory extraction task **still holds references to the state at the end of the conversation** (e.g., conversation context, pending operations, etc.). These states should have been cleaned up when the session closed, but because the extraction task is still running, they are "closed over"
> - The essence is: **background tasks hold references to a closed session's context, preventing it from being garbage collected**

18.4 Submission Pipeline

```
executeExtractMemories() triggers
  → Check GrowthBook gate (tengu_passport_quail)
  → Check isAutoMemoryEnabled()
  → Check inProgress (decide whether to stash for trailing)
  → Call scanMemoryFiles() to get manifest (inject early to avoid agent wasting a turn on
ls)
  → Build extraction prompt (with newMessageCount + existingMemories)
  → runForkedAgent({
      canUseTool: createAutoMemCanUseTool(memoryDir),
      maxTurns: 5,
      skipTranscript: true,
    })
  → Extract writtenPaths
  → appendSystemMessage({ "Saved N memories" }) to main conversation
  → Refresh cursor lastMemoryMessageUuid

drainPendingExtraction() — called at graceful shutdown
  → Wait for all inFlightExtractions to complete (max 60s)
  → Prevent shutdown from killing forked agents currently running
```

18.5 Extraction Prompt Template

Two prompt variants, controlled by `tengu_moth_copse` GrowthBook flag for whether to skip index writing:

Auto-Only variant (`buildExtractAutoOnlyPrompt`):

- Extracts **private personal memories only**, saves to local directory
- Tells main agent: if user asks to remember something, save it immediately
- Provides four type definitions (user/feedback/project/reference)
- Provides "What NOT to save" list
- Two-step save: ① write topic file ② update MEMORY.md (index)

Combined variant (`buildExtractCombinedPrompt` , team-only feature):

- Adds **team memory** on top of Auto-Only
- Extra `<scope>` tags for each type (private vs. team)
- Extra requirement: don't save sensitive data in team memory
- Two-step save: each for private and team directories separately

19. DreamTask (Dream Task UI Integration)

19.1 Design Background

The Auto-Dream forked agent is invisible to the user by default. DreamTask integrates it into the UI through the task registration framework (`registerTask` / `updateTaskState`), enabling the background process to display in the footer pill and Shift+Down dialog.

19.2 Task State Structure

```
type DreamTaskState = {
  type: 'dream'            // Task type identifier
  status: 'running' | 'completed' | 'failed' | 'killed'
  phase: 'starting' | 'updating'
  sessionsReviewing: number
  filesTouched: string[]  // At least modified file paths (incomplete)
  turns: DreamTurn[]      // Last 30 assistant responses
  priorMtime: number      // Used for rollback after kill
}
```

19.3 Message Listener (makeDreamProgressWatcher)

`makeDreamProgressWatcher` in autoDream.ts listens to each assistant message from the forked agent: Whenever the forked agent sends an assistant message, immediately extract three things:

- Collect text blocks into turn.text
- Count tool_use block quantity
- Extract file_path from Edit/Write tool_use into filesTouched

> **Note**
>
> Because Auto Dream is a background fork process, users can't see its complete output, but it edits files. The operation here is to produce a structured summary informing the user "what Auto Dream did" — collecting progress in real time, summarizing for display after completion

19.4 Kill Task

Killing Dream from UI:

1. `abortController.abort()` — abort API call
2. `rollbackConsolidationLock(priorMtime)` — rollback lock mtime, enabling retry next time

---

20. Daily Log Mode (Assistant Mode / KAIROS)

20.1 Background

In Assistant mode (perpetual session, as opposed to CLI mode which runs separately each time), **conversations are continuous and cannot rely on conversation pause points**. Claude Code designed the **Daily Log** mode for this.

20.2 Differences from Standard Mode

Feature	Standard Mode (CLI)	Daily Log Mode (Assistant)
New memory destination	Directly write to topic file	Append to `logs/YYYY/MM/YYYY-MM-DD.md`
MEMORY.md maintenance	Update index after each memory write	暂时不维护（log 是 append-only）
Index organization	None	Periodically distill logs into topic files via nightly /dream skill
Trigger condition	—	feature('KAIROS')

Note

- Non-KAIROS mode: uses Auto Dream (forked agent runs memory consolidation in background)
- KAIROS mode: uses disk-skill dream, Auto Dream is skipped

---

21. Startup Chain and Module Initialization

21.1 backgroundHousekeeping.ts

All memory modules are initialized in `backgroundHousekeeping.ts` (in parallel with `initExtractMemories`):

```
export function startBackgroundHousekeeping(): void {
  // ① Always initialize (unconditional)
  void initMagicDocs()           // Magic Docs service
  void initSkillImprovement()    // Skill improvement

  // ② Conditional compile (tree-shaken if EXTRACT_MEMORIES not enabled)
  if (feature('EXTRACT_MEMORIES')) {
    extractMemoriesModule!.initExtractMemories()
  }

  // ③ Always initialize (after extractMemories)
  initAutoDream()

  // ④ Plugin auto-update
  void autoUpdateMarketplacesAndPluginsInBackground()

  // ⑤ Deep link registration (interactive mode + LODESTONE only)
  if (feature('LODESTONE') && getIsInteractive()) {
    void registerProtocolModule!.ensureDeepLinkProtocolRegistered()
  }

  // ⑥ Delay slow operations (execute after 10 minutes)
  setTimeout(runVerySlowOps, 10min).unref()

  // ⑦ ant users: periodic cleanup every 24 hours
  if (process.env.USER_TYPE === 'ant') {
    setInterval(cleanupNpmCache + cleanupOldVersions, 24h).unref()
  }
}
```

backgroundHousekeeping.ts is a **background task housekeeper** pulled up in parallel at startup: initializes memory/docs/Dream services, schedules cleanup tasks, and does not block main process startup throughout.

> **Note**
>
> - Delayed slow operations (runVerySlowOps): During startup, if the user is already operating, cleanup tasks should not compete for resources. Wait until the user is idle before running.
> - .unref() — does not block process exit: these timers are not counted as "active references" in Node.js. Process exit is not blocked by these timers.

21.2 stopHooks.ts Trigger Chain

```
// src/query/stopHooks.ts
// Triggers at end of each conversation turn (model outputs final response, no tool call):
void executeExtractMemories(stopHookContext, appendSystemMessage)
void executeAutoDream(stopHookContext, appendSystemMessage)
```

Both systems are fire-and-forget (do not await completion), so they run in parallel non-blocking.

21.3 Drain on Shutdown

```
// print.ts — after response flush, before gracefulShutdownSync
await drainPendingExtraction(60_000)  // wait max 60s
```

Appendix A: Zod Schema Definitions

A.1 Schema Definitions

```
// teamMemorySync/types.ts
const TeamMemoryContentSchema = z.object({
  entries: z.record(z.string(), z.string()),           // key -> content
  entryChecksums: z.record(z.string(), z.string()).optional(),
})

const TeamMemoryDataSchema = z.object({
  organizationId: z.string(),
  repo: z.string(),
  version: z.number(),
  lastModified: z.string(),
  checksum: z.string(),
  content: TeamMemoryContentSchema(),
})

// agentMemorySnapshot.ts
const snapshotMetaSchema = z.object({
  updatedAt: z.string().min(1),
})

const syncedMetaSchema = z.object({
```

```
  syncedFrom: z.string().min(1),
})
```

Appendix B: Global State Overview

B.1 Global State per Module

Session memory module maintains the following global state (`sessionMemoryUtils.ts`):

```
let lastSummarizedMessageId: string | undefined = undefined
let extractionStartedAt: number | undefined = undefined
let tokensAtLastExtraction = 0
let sessionMemoryInitialized = false
let sessionMemoryConfig: SessionMemoryConfig = { ...DEFAULT_SESSION_MEMORY_CONFIG }
```

Watcher module maintains the following global state (`watcher.ts`):

```
let watcher: FSWatcher | null = null
let debounceTimer: ReturnType<typeof setTimeout> | null = null
let pushInProgress = false
let hasPendingChanges = false
let currentPushPromise: Promise<void> | null = null
let watcherStarted = false
let pushSuppressedReason: string | null = null
let syncState: SyncState | null = null
```

ExtractMemories module maintains the following global state (`extractMemories.ts` , closure scope):

```
let lastMemoryMessageUuid: string | undefined        // Cursor: UUID of last processed
message
let inProgress = false                               // Whether an extraction is currently
running
let turnsSinceLastExtraction = 0                    // Throttle counter
let pendingContext: { context, appendSystemMessage } | undefined  // Trailing run backlog
context
let inFlightExtractions = new Set<Promise<void>>()  // All pending extraction Promises
let hasLoggedGateFailure = false                    // Whether gate-disabled log has been
recorded
```

AutoDream module maintains the following global state (`autoDream.ts` , closure scope):

```
let runner: ((context, appendSystemMessage?) => Promise<void>) | null = null
let lastSessionScanAt = 0  // Last scan time (for SESSION_SCAN_INTERVAL_MS throttling)
```